

UNIVERSIDADE DA BEIRA INTERIOR
Covilhã | Portugal

Relatório do Trabalho Prático 2 - Farmácia
GUI

Afonso Esteves - 55047
Felipe Dourado - 53394
Tiago Pereira - 55019

6 de novembro de 2025

Conteúdo

Resumo **3**

1 Introdução **4**
1.1 Âmbito, enquadramento e motivação 4

2 Contextualização do Projeto **4**
2.1 Identificação dos Desenvolvedores 4
2.1.1 Desenvolvedores Envolvidos 4
2.2 Contextualização Académica 4

3 Especificações Técnicas **5**
3.1 Ambiente de Desenvolvimento 5

4 Utilização **5**
4.1 Requisitos Prévios 5
4.2 Compilação e Execução 5
4.3 Execução de Testes Unitários 6
4.4 Geração de Documentação Javadoc 6

5 Estrutura da Solução Construída **6**
5.1 Componentes do Sistema 6
5.2 Testes e Documentação 7

6 Funcionalidades Implementadas **7**

7 Implementação de Funcionalidades **8**
7.1 Classe FarmaciaGUI: A Janela Principal 8
7.2 Classe GestorDados: Persistência de Dados 8
7.3 Classe FarmaciaTest: Testes Unitários 9

8 Considerações de Implementação **10**
8.1 Princípios de Desenvolvimento 10

9 Conclusão **10**

10 Referências Bibliográficas **11**

Resumo

Este relatório documenta a evolução de uma aplicação de gestão de farmácia, migrando de uma interface de consola para uma interface gráfica de utilizador (GUI) desenvolvida com Java Swing. O sistema foi reestruturado em pacotes (modelo, GUI, persistência) para melhor organização e manutenibilidade, seguindo princípios de arquitetura em camadas. Além da nova interface, foi implementada a persistência de dados através de serialização de objetos, permitindo que o estado da aplicação (clientes, produtos e encomendas) seja guardado e recuperado entre sessões. Esta segunda versão representa um avanço significativo em termos de usabilidade e robustez.

Palavras-chave e *Keywords*

Java; Swing; GUI; Programação Orientada a Objetos; Persistência de Dados; Serialização; JUnit; Javadoc.

1 Introdução

1.1 Âmbito, enquadramento e motivação

Este documento detalha o desenvolvimento do Trabalho Prático 2, uma evolução do sistema de gestão de farmácia introduzido no TP1. O foco principal desta nova fase foi a substituição da interface de linha de comandos por uma interface gráfica (GUI) mais intuitiva e funcional, utilizando a biblioteca Java Swing. Adicionalmente, foi introduzida uma camada de persistência para garantir que os dados da aplicação não são perdidos ao fechar o programa, e foram adicionados testes unitários e documentação formal ao projeto.

A motivação foi aprofundar os conhecimentos em Programação Orientada a Objetos, explorando conceitos mais avançados como o desenvolvimento de interfaces gráficas, programação orientada a eventos, serialização de objetos, testes com JUnit, documentação com Javadoc e a importância de uma arquitetura de software bem definida.

2 Contextualização do Projeto

2.1 Identificação dos Desenvolvedores

Os autores do presente projeto são estudantes do curso de Informática Web, Móvel e na Nuvem da Universidade da Beira Interior (UBI), matriculados na disciplina de Programação Orientada a Objetos.

2.1.1 Desenvolvedores Envolvidos

- **Afonso Esteves (55047)**
- **Felipe Dourado (53394)**
- **Tiago Pereira (55019)**

2.2 Contextualização Académica

O projeto foi desenvolvido no contexto da disciplina de Programação Orientada a Objetos, lecionada pelo Professor Paul Andrew Crocker, com o objetivo de aplicar e consolidar conhecimentos em:

- Desenvolvimento de Interfaces Gráficas (GUI) com Java Swing.
- Programação Orientada a Eventos (ActionListeners, MouseListeners).

- Persistência de dados através de serialização de objetos Java.
- Arquitetura de software em camadas (Modelo, Visão, Persistência).
- Testes unitários com JUnit para garantir a qualidade do código.
- Geração de documentação técnica com Javadoc.
- Refatoração e melhoria de código existente.

3 Especificações Técnicas

3.1 Ambiente de Desenvolvimento

- **Linguagem:** Java (JDK 8 ou superior)
- **Bibliotecas Gráficas:** Java Swing
- **Testes:** JUnit 5
- **Documentação:** Javadoc
- **Ambiente de Execução:** Java Development Kit (JDK)
- **Ambiente de Desenvolvimento:** IntelliJ IDEA

4 Utilização

4.1 Requisitos Prévios

- **Java Development Kit (JDK) 8 ou superior:** Essencial para compilar e executar a aplicação. Pode ser descarregado do site oficial da Oracle.

4.2 Compilação e Execução

1. **Navegar até ao diretório raiz do projeto.**
2. **Compilar os ficheiros Java:**

```
javac -d TP-02/out/production/TP-02 -cp TP-02/lib/
junit-platform-console-standalone-1.10.2.jar
$(find TP-02/src -name "*.java")
```

3. **Executar a aplicação GUI:**

   ```
   java -cp TP-02/out/production/TP-02 Main
   ```

4.3 Execução de Testes Unitários

Para verificar a integridade da lógica de negócio, podem ser executados os testes JUnit 5:

1. **Navegar até ao diretório raiz do projeto.**
2. **Executar os testes (após compilação):**

   ```
   java -jar TP-02/lib/junit-platform-console-standalone-1.10.2.jar -cp TP-02/c
   ```

4.4 Geração de Documentação Javadoc

Para gerar a documentação HTML do código fonte:

1. **Navegar até ao diretório raiz do projeto.**
2. **Gerar a documentação:**

   ```
   javadoc -d TP-02/doc TP-02/src/**/*.java TP-02/src/*.java
   ```

   A documentação ficará disponível no diretório 'TP-02/doc/'.

5 Estrutura da Solução Construída

O sistema foi reestruturado para uma arquitetura em camadas, separando as responsabilidades em pacotes distintos.

5.1 Componentes do Sistema

- **Pacote 'modelo':** Contém as classes que representam o domínio do problema (`Cliente`, `Produto`, `Farmacia`, etc.).
- **Pacote 'gui':** Contém as classes da interface gráfica (`FarmaciaGUI`, `PainelGestaoClientes`, etc.).

- **Pacote ‘persistencia‘:** Contém a classe `GestorDados`, responsável por guardar e carregar o estado da aplicação.
- **Pacote ‘test‘:** Contém as classes de teste unitário, como `FarmaciaTest.java`.
- **Classe ‘Main‘:** Ponto de entrada que inicializa a farmácia, carrega os dados e lança a GUI.

5.2 Testes e Documentação

O projeto inclui artefactos importantes para garantir a qualidade e a manutenibilidade do software:

- **Testes Unitários:** O diretório ‘TP-02/src/test/‘ contém a classe ‘FarmaciaTest.java‘, que utiliza o framework JUnit 5 para validar a lógica de negócio da classe ‘Farmacia‘, como adicionar clientes, gerir stock e processar encomendas.
- **Documentação Javadoc:** O diretório ‘TP-02/doc/‘ é o destino da documentação gerada pela ferramenta Javadoc. Esta documentação descreve as classes, métodos e a API do projeto, facilitando a sua compreensão e reutilização.

6 Funcionalidades Implementadas

As funcionalidades do sistema foram expandidas e agora são acedidas através da interface gráfica:

- **Interface Gráfica com Separadores:** A janela principal usa um `JTabbedPane` para separar a gestão de Clientes, Produtos e Encomendas.
- **Gestão CRUD Completa:** Todas as entidades podem ser criadas, lidas, atualizadas e removidas (CRUD) através de formulários e tabelas interativas.
- **Persistência Automática:** Os dados da aplicação são automaticamente carregados ao iniciar e guardados ao fechar a janela.
- **Testes Unitários:** Cobertura de testes para a lógica de negócio principal, garantindo robustez.

7 Implementação de Funcionalidades

7.1 Classe FarmaciaGUI: A Janela Principal

A classe 'FarmaciaGUI' configura a janela principal, o 'JTabbedPane' e o comportamento de fecho para guardar os dados.

```
public class FarmaciaGUI extends JFrame {
    // ...
    public FarmaciaGUI(Farmacia farmacia) {
        // ...
        setTitle("Gest o de Farm cia");
        setSize(800, 600);
        setDefaultCloseOperation(JFrame.DO_NOTHING_ON_CLOSE);
        setLocationRelativeTo(null);

        addWindowListener(new WindowAdapter() {
            @Override
            public void windowClosing(WindowEvent e) {
                // ... L gica para guardar dados ...
            }
        });

        JTabbedPane tabbedPane = new JTabbedPane();
        // ... cria  o e adi  o dos pain is ...
        add(tabbedPane);
    }
}
```

Listing 1: Configuração da janela principal em FarmaciaGUI.java

7.2 Classe GestorDados: Persistência de Dados

A classe 'GestorDados' usa serialização para guardar e carregar as listas de objetos que compõem o estado da aplicação.

```java
public static void guardarDados(List<Cliente> clientes,
    List<Produto> produtos, List<Encomenda> encomendas) {
    try (ObjectOutputStream oos = new ObjectOutputStream(new
        FileOutputStream(NOME_FICHEIRO))) {
        oos.writeObject(clientes);
        oos.writeObject(produtos);
        oos.writeObject(encomendas);
    } catch (IOException e) {
        // ...
    }
}
```

Listing 2: Método para guardar dados em GestorDados.java

7.3 Classe FarmaciaTest: Testes Unitários

Os testes com JUnit 5 validam o comportamento esperado da lógica de negócio, como a adição de um cliente.

```java
class FarmaciaTest {
    private Farmacia farmacia;

    @BeforeEach
    void setUp() {
        farmacia = new Farmacia();
    }

    @Test
    void testAdicionarCliente() {
        Cliente cliente = new Cliente("Nome Teste", "123456789",
            new Morada("Rua Teste", "Cidade Teste", "6200-000"));
        farmacia.adicionarCliente(cliente);
        assertEquals(1, farmacia.getClientes().size());
        assertEquals("Nome Teste",
            farmacia.acharClienteNif("123456789").get().getNome());
    }
}
```

Listing 3: Exemplo de um teste unitário em FarmaciaTest.java

8 Considerações de Implementação

8.1 Princípios de Desenvolvimento

- **Arquitetura MVC (Model-View-Controller):** A estrutura em pacotes 'modelo' (Model), 'gui' (View) e a lógica de eventos nos painéis (Controller) aproxima o projeto do padrão MVC.
- **Programação Orientada a Eventos:** A GUI é controlada por eventos gerados pelo utilizador, tratados por 'Listeners'.
- **Test-Driven Development (TDD):** A inclusão de testes unitários com JUnit incentiva um ciclo de desenvolvimento onde os testes guiam a implementação da lógica de negócio.

9 Conclusão

O Trabalho Prático 2 representa um marco importante na evolução do sistema de farmácia. A migração para uma interface gráfica com Swing melhorou a usabilidade, enquanto a implementação da persistência de dados com serialização tornou a aplicação mais robusta. A adição de testes unitários com JUnit e a geração de documentação com Javadoc elevaram a qualidade e a profissionalismo do projeto, solidificando os conhecimentos em engenharia de software e Programação Orientada a Objetos.

10 Referências Bibliográficas

- Schildt, Herbert. *Java: A Beginner's Guide*. 6th ed., McGraw-Hill Education, 2014.
- Olsson, Mikael. *Java Quick Syntax Reference*. Apress, 2014.
- Roberts, Eric S. *The Art and Science of Java*. Addison-Wesley, 2008.

Acrónimos

UBI Universidade da Beira Interior